

```

import time
import requests
import os

from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.chrome.options import Options
from selenium.webdriver.chrome.service import Service
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

# =====
# CONFIG
# =====

TOKEN = os.getenv("TOKEN")
CHAT_ID = os.getenv("CHAT_ID")

# URL directo a High Rollers de deportes
URL = "https://stake1017.com/sports/high-rollers?c=playstakeio"

# =====
# TELEGRAM
# =====

def enviar_mensaje(msg):
    url = f"https://api.telegram.org/bot{TOKEN}/sendMessage"
    try:
        requests.post(url, data={
            "chat_id": CHAT_ID,
            "text": msg
        })
    except Exception as e:
        print("Error enviando mensaje Telegram:", e)

# =====
# SELENIUM SETUP
# =====

options = Options()
options.add_argument("--headless=new")
options.add_argument("--no-sandbox")
options.add_argument("--disable-dev-shm-usage")
options.add_argument("--disable-gpu")

```

```

options.add_argument("--disable-software-rasterizer")
options.add_argument("--window-size=1920,1080")
options.add_argument("--user-agent=Mozilla/5.0 (Windows NT 10.0; Win64; x64) Appl
options.binary_location = "/usr/bin/chromium"

```

```

driver = webdriver.Chrome(
    service=Service("/usr/bin/chromedriver"),
    options=options
)

```

```

# =====
# ESTADO
# =====

```

```

vistos = set()

```

```

# =====
# HELPERS
# =====

```

```

def parsear_monto(texto):
    try:
        texto = texto.replace("$", "").replace(",", "").strip()
        texto = ''.join(c for c in texto if c.isdigit() or c == '.')
        return float(texto) if texto else 0
    except:
        return 0

```

```

def clasificar_monto(monto):
    if monto >= 100000:
        return "HUGE", "🟡"
    elif monto >= 50000:
        return "BIG", "🔴"
    elif monto >= 10000:
        return "MEDIUM", "🟠"
    elif monto >= 3000:
        return "SMALL", "🟡"
    else:
        return None, None

```

```

# =====
# SCRAPING
# =====

```

```

def obtener_apuestas():

```

```

try:
    driver.get(URL)
    print("URL actual:", driver.current_url)

    # Esperar a que cargue alguna fila
    try:
        WebDriverWait(driver, 15).until(
            EC.presence_of_element_located((By.XPATH, "//*[contains(@class,'r
            )
    except:
        print("Timeout esperando filas")

    time.sleep(5)

    # Imprimir HTML para debug (primeros 3000 chars)
    html = driver.page_source
    print("HTML snippet:", html[:3000])

except Exception as e:
    print("Error cargando página:", e)
    return []

apuestas = []

# Intentar múltiples estrategias de XPath
estrategias = [
    "//div[contains(@class,'table')]/div[contains(@class,'row')]",
    "//div[contains(@class,'bets')]/div[contains(@class,'item')]",
    "//div[contains(@class,'high-roller')]/div[contains(@class,'row')]",
    "//tbody/tr",
    "//div[@data-testid='bet-row']",
    "//*[contains(@class,'bet-row')]",
    "//*[contains(@class,'wager')]",
]

filas = []
for xpath in estrategias:
    filas = driver.find_elements(By.XPATH, xpath)
    if filas:
        print(f"XPath funcionó: {xpath} - {len(filas)} filas")
        break

print("Filas encontradas:", len(filas))

for fila in filas:
    try:
        texto = fila.text.strip()

```

```

        if not texto:
            continue

        columnas = texto.split("\n")
        print("Columnas:", columnas)

        if len(columnas) < 2:
            continue

        evento = columnas[0]

        # Buscar el monto: la columna que contenga "$"
        monto_texto = ""
        cuota = ""
        for col in reversed(columnas):
            if "$" in col and not monto_texto:
                monto_texto = col
            elif monto_texto and not cuota:
                cuota = col
            break

        if not monto_texto:
            monto_texto = columnas[-1]

        monto = parsear_monto(monto_texto)

        apuestas.append({
            "evento": evento,
            "cuota": cuota,
            "monto": monto,
            "raw": texto
        })

    except Exception as e:
        print("Error parseando fila:", e)
        continue

    return apuestas

# =====
# LOOP PRINCIPAL
# =====

print("🚀 Bot iniciado...")

while True:

```

```

try:
    apuestas = obtener_apuestas()

    for a in apuestas:
        key = a["raw"]

        if key in vistos:
            continue

        vistos.add(key)

        categoria, emoji = clasificar_monto(a["monto"])

        if categoria is None:
            continue

        msg = f""{emoji} {categoria} BET DETECTED

🎯 Evento: {a['evento']}
💰 Monto: ${a['monto']:,.2f}
📊 Cuota: {a['cuota']}
"""

        enviar_mensaje(msg)
        print(f"✅ Enviado: {categoria} - {a['evento']} - ${a['monto']}")

        time.sleep(15)

except Exception as e:
    print("ERROR GENERAL:", e)
    time.sleep(10)

```